

tf.custom_gradient Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/custom_gradient

Decorator to define a function with a custom gradient.

Function Signatures

```
tf.custom_gradient( f=None )  
def log1pexp(x): return tf.math.log(1 + tf.exp(x))  
with tf.GradientTape() as tape: tape.watch(x) y=log1pexp(x)  
dy_dx = tape.gradient(y, x) # Will be NaN when evaluated.  
@tf.custom_gradient def log1pexp(x): e = tf.exp(x) def  
grad(upstream): return upstream * (1 - 1 / (1 + e)) return  
tf.math.log(1 + e), grad
```

Code Examples

```
tf.custom_gradient(  
f=None  
)
```

```
tf.custom_gradient(  
f=None  
)
```

```
def log1pexp(x):  
    return tf.math.log(1 + tf.exp(x))
```

```
def log1pexp(x):  
    return tf.math.log(1 + tf.exp(x))
```

```
with tf.GradientTape() as tape:  
    tape.watch(x)  
    y=log1pexp(x)  
    dy_dx = tape.gradient(y, x) # Will be NaN when evaluated.
```

```
with tf.GradientTape() as tape:  
    tape.watch(x)  
    y=log1pexp(x)  
    dy_dx = tape.gradient(y, x) # Will be NaN when evaluated.
```

```
@tf.custom_gradient  
def log1pexp(x):  
    e = tf.exp(x)  
    def grad(upstream):  
        return upstream * (1 - 1 / (1 + e))  
    return tf.math.log(1 + e), grad
```

```
@tf.custom_gradient  
def log1pexp(x):  
    e = tf.exp(x)  
    def grad(upstream):  
        return upstream * (1 - 1 / (1 + e))  
    return tf.math.log(1 + e), grad
```

Documentation

Decorator to define a function with a custom gradient.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.custom_gradient`

Used in the notebooks

- Advanced automatic differentiation

This decorator allows fine grained control over the gradients of a sequence for operations. This may be useful for multiple reasons, including providing a more efficient or numerically stable gradient for a sequence of operations.

For example, consider the following function that commonly occurs in the computation of cross entropy and log likelihoods:

Due to numerical instability, the gradient of this function evaluated at $x=100$ is NaN. For example:

The gradient expression can be analytically simplified to provide numerical stability:

With this definition, the gradient dy_dx at $x = 100$ will be correctly evaluated as 1.0.

The variable upstream is defined as the upstream gradient. i.e. the gradient from all the layers or functions originating from this layer. The above example has no upstream functions, therefore $\text{upstream} = dy/dy = 1.0$.

Assume that x_i is $\log 1 + \exp$ in the forward pass $x_1 = x_1(x_0)$, $x_2 = x_2(x_1)$, ..., $x_i = x_i(x_{i-1})$, ..., $x_n = x_n(x_{n-1})$. By chain rule we know that $dx_n/dx_0 = dx_n/dx_{n-1} * dx_{n-1}/dx_{n-2} * \dots * dx_i/dx_{i-1} * \dots * dx_1/dx_0$.

In this case the gradient of our current function defined as $dx_i/dx_{i-1} = (\exp(x_i) / (1 + \exp(x_i))) = (1 - 1 / (1 + \exp(x_i)))$. The upstream gradient upstream would be $dx_n/dx_{n-1} * dx_{n-1}/dx_{n-2} * \dots * dx_{i+1}/dx_i$. The upstream gradient multiplied by the current gradient is then passed downstream.

In case the function takes multiple variables as input, the grad function must also return the same number of variables.

We take the function $z = x * y$ as an example.

Nesting custom gradients can lead to unintuitive results. The default behavior does not correspond to n-th order derivatives. For example

The function `grad_grad_fn` will be calculating the first order gradient of `grad_fn` with respect to `dy`, which is used to generate forward-mode gradient graphs from backward-mode gradient graphs, but is not the same as the second order gradient of `op` with respect to `x`.

Instead, wrap nested `@tf.custom_gradients` in another function:

Additional arguments to the inner `@tf.custom_gradient`-decorated function control the expected return values of the innermost function.

The examples above illustrate how to specify custom gradients for functions which do not read from variables. The following example uses variables, which require special handling because they are effectively inputs of the forward function.

Above example illustrates usage of trainable variable weights.

In the example, the inner `grad_fn` accepts an extra variables input parameter and also returns an extra `grad_vars` output. That extra argument is passed if the forward function reads any variables. You need to compute the gradient w.r.t. each of those variables and output it as a list of `grad_vars`. Note here that default value of variables is set to `None` when no variables are used in the forward function.

It should be noted `tf.GradientTape` is still watching the forward pass of a `tf.custom_gradient`, and will use the ops it watches. As a consequence, calling `tf.function` while the tape is still watching leads to a gradient graph being built. If an op is used in `tf.function` without registered gradient, a `LookupError` will be raised.

Users can insert `tf.stop_gradient` to customize this behavior. This is demonstrated in the example below. `tf.random.shuffle` does not have a registered gradient. As a result `tf.stop_gradient` is used to avoid the `LookupError`.

See also `tf.RegisterGradient` which registers a gradient function for a primitive TensorFlow operation. `tf.custom_gradient` on the other hand allows for fine grained control over the gradient computation of a sequence of operations.

Note that if the decorated function uses `Variables`, the enclosing variable scope must be using `ResourceVariables`.

`## Returns`